CS 236: Operating Systems Lab
Spring 2025-26
Lab 7: no schedule, no progress!

xv6 is a teaching operating system (developed at MIT) — an OS meant for teaching design and implementation concepts of an OS.

To install xv6 on Ubuntu, look up the details described in **Appendix A.**

The Holy Grail for xv6

- xv6-risc book
- xv6-risc src booklet

The xv6 source tree is organized to mirror OS subsystems:

- **`kernel/`** → process management, memory, file system, traps
- **`user/`** → user programs
- **`mkfs/`** → file system creation tool

Each directory corresponds to a logical OS component, enabling systematic exploration and modification. **You will mostly interact with the `kernel/` and `user/` directory.**

Submission Guidelines

Create a patch with:

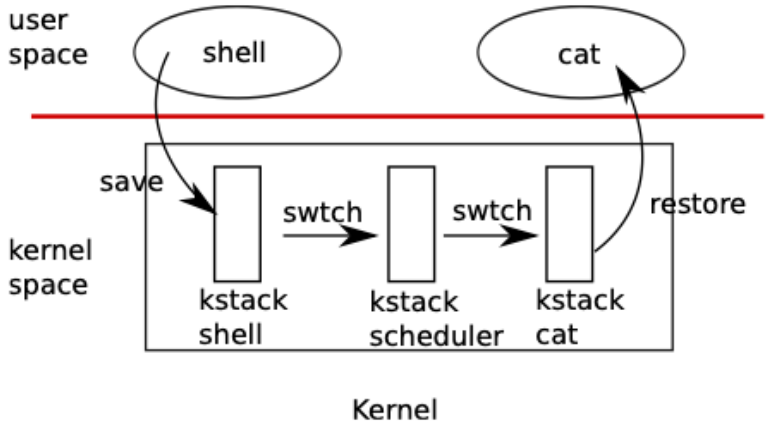
```
Shell
❯ ./patch.sh
```

Rename the generated **lab7.patch** to **`<rollno>_lab7.patch`** and submit via Moodle.

in the context of the kernel

Most times, the kernel executes in the context of a process—the process context for the kernel to execute system calls etc. on behalf of the process. The execution state of the process is stored and restored on the trapframe.

While executing in the kernel mode, the kernel may decide to invoke the scheduler and potentially schedule a different process. At this point, the kernel has to save and restore the execution in the kernel for the outgoing —referred to as **context** by xv6 and stored in the struct proc (PCB) of each process. The **context** variable is used to save and restore the runtime state in the kernel for the process and is the save and restore state of a process and the kernel when scheduling.



"xv6 does not directly context switch from one process's kernel thread to another process's kernel thread; instead, a kernel thread gives up the CPU by context switching to that CPU's "scheduler thread," and the scheduler thread picks a different process's kernel thread to run, and context-switches to that thread. "
`reference: Ch. 8 [xv6book]`

1. what is the deal with `forkret`?

As a first step, let's understand the role of the xv6 function **`forkret`**. To do so, print the values of the two variables (**`ra`** and **`sp`**) of the context structure stored in the PCB of the process. Do so while executing the **`forkret`** function. The output format should look like the one shown below.

1a. What does the value of variable **`ra`** correspond to? Print another value to verify your answer.

1b. How is the kernel stack pointer value (**`sp`**) different for each process? E.g., allocproc does not configure the kstack value while allocation and initializing the PCB.

1c. Where and how are values in the **`ra`** and **`sp`** variables and registers updated/used outside of the function **`forkret`**?

Sample Output

```
Shell
xv6 kernel is booting

[pid 1] ppid=NA ra=0x0000000080001912 kstack=0x0000003fffffd000
init: starting sh
[pid 2] ppid=1 ra=0x0000000080001912 kstack=0x0000003fffffb000
$ echo hello cs236
[pid 3] ppid=2 ra=0x0000000080001912 kstack=0x0000003fffff9000
hello cs236
```

2. switcheroo!

The scheduling magic in xv6 is via three main functions — **`sched`**, **`swtch`** and **`scheduler`**

In fact the **`scheduler`** function (the **only** function of each per-cpu kernel thread) is only involved **once**, but still seems to keep scheduling processes. Whenever the scheduler chooses a process to schedule, an user process has already been **switched out** (scheduled out) and the scheduler **switches in** the chosen user process (to be scheduled in).
Check the **`sched`** and **`swtch`** implementation and also details of **`struct cpu`**.

As part of this question, count the number of context switch events — switch outs and switch ins for each process.

- Modify the **`struct proc`** to add two new variables
  `switch_ins[NCPU], switch_outs[NCPU]`,
  to store the number of times a process is switched in/out on each CPU.
- Modify the appropriate scheduling related functions to count the switch in and switch out events per process.
- Implement the following system call to read out the values into the user space for the corresponding process. The call returns the set of values across all CPUs for the calling process.
  `int getschedstatus(int* switchin, int* switchout);`

Sample Output (set CPUS = 1)

```
Shell
$ p2
-----------------------------------
```

```
RUNNING: single busy process for 20 ticks
busy-A (pid=4) in=21 out=20
-----------------------------------
RUNNING: single sleep process for 20 ticks
sleep-A (pid=5) in=21 out=20
-----------------------------------
RUNNING: two BUSY processes for 20 ticks
busy-A (pid=6) in=11 out=10
busy-B (pid=7) in=12 out=11
-----------------------------------
RUNNING: BUSY + SLEEP process for 20 ticks
busy-A (pid=8) in=21 out=20
sleep-B (pid=9) in=21 out=20
-----------------------------------
RUNNING: two SLEEP processes for 20 ticks
sleep-A (pid=10) in=21 out=20
sleep-B (pid=11) in=21 out=20
-----------------------------------
```

Why are more context switches reported when running a busy process and a sleeping process (or two sleeping processes) as compared to two busy processes in parallel ?

Hint: Look up the implementation of **`sys_pause`** and **`sleep`** in the kernel.

Sample Output (set CPUS = 2)

Shell

```
$ p2
-----------------------------------
RUNNING: single busy process for 20 ticks
busy-A (pid=4) in=21 out=20
-----------------------------------
RUNNING: single sleep process for 20 ticks
sleep-A (pid=5) in=21 out=20
-----------------------------------
RUNNING: two BUSY processes for 20 ticks
busy-A (pid=6) in=21 out=20
busy-B (pid=7) in=20 out=19
-----------------------------------
RUNNING: BUSY + SLEEP process for 20 ticks
busy-A (pid=8) in=21 out=20
sleep-B (pid=9) in=21 out=20
```

```
-------------------------------------
RUNNING: two SLEEP processes for 20 ticks
sleep-A (pid=10) in=21 out=20
sleep-B (pid=11) in=21 out=20
-------------------------------------
```

3. last call for priority passengers

As part of this question, update the scheduler's scheduling logic to support the notion of priority — processes with higher priority are scheduled before processes with lower priority. We will work with two priority levels and the core logic of the default round robin scheduling will remain the same. The logic will be used with higher priority processes as long as they are READY and then applied to the non-higher priority processes.

Tasks

- Set CPUS = 1 in the **`Makefile`**
- Implement a syscall to enable/disable priority scheduling,
  E.g., arg is 0 to disable, arg is 1 to enable.
  `int setprioritysched(int enable)`
- Implement a syscall to set priority of a process for scheduling.
  `int setpriority(int priority)`
- Implement the priority scheduling logic in the scheduler
- Measure time taken/duration for execution of processes with/without priority

Sample Output

Shell

```
$ p3
=== Round Robin Scheduling ===
[pid 4] start: 23, end: 59
[pid 5] start: 23, end: 60

=== Priority Scheduling===
[pid 6] setting high priority
[pid 7] setting low priority
[pid 6] start: 60, end: 79
[pid 7] start: 60, end: 97
```

4. i looooooove this cpu!

As the last part of this lab, implement the notion of CPU affinity — a per-process information, which specifies which CPU or CPUs can a process be scheduled.

Tasks

- Implement a system call to pin a process to a CPU core.
  `int setcpuaffinity(int flag);`
  flag is specified as a bit vector which is NCPU's wide, with the 0th bit mapping to CPU0 and bit 1 to CPU1 etc.
  A 1 bit indicates affinity to the CPU and a 0 indicates otherwise.
  To schedule a process on any of the available CPUs, set all bits of the flag to 1.
- Note that affinity is a per process setting.
- Implement logic in the scheduler to honour CPU affinity.
- Use **p4.c** to check for correctness.
- Set CPUS = 2 for the sample output.

Note: The `p4.c` program executes 4 concurrent child processes but only displays the scheduling statistics (In/Out counts) for **Child 1**.

Sample Output (CPUs = 2)

```
Shell
$ p4
=== Scenario: Without CPU Affinity ===
-> stats for child1:
CPU 0: in=25 out=25
CPU 1: in=26 out=25

=== Scenario: child1(CPU 0) others(CPU 1) ===
-> stats for child1:
CPU 0: in=99 out=98
CPU 1: in=1 out=1

=== Scenario: child1(CPU 0 & 1) others(CPU 1) ===
-> stats for child1:
CPU 0: in=98 out=97
CPU 1: in=2 out=2

=== Scenario: child1(CPU 0 & 1) others(CPU 0 & 1) ===
-> stats for child1:
CPU 0: in=25 out=25
CPU 1: in=26 out=25
```

Appendix A

Follow these steps for installing xv6 on a Ubuntu-based Linux machine:

1. Prerequisites
    a. First, update your system:
    `sudo apt update`
    b. Install the required packages:
    `sudo apt install -y git build-essential qemu-system-riscv64 gcc-riscv64-unknown-elf`
    - **`build-essential`**
      For essential packages like libc, gcc, make, etc.
    - **`qemu-system-riscv64`**
      Emulates a RISC-V machine in software.
      Your computer's real CPU speaks x86-64, but xv6 is written for RISC-V.
      QEMU **pretends to be a RISC-V computer,** so the OS can run even though the hardware is different.
    - **`gcc-riscv64-unknown-elf`**
      It runs on your x86-64 machine but produces **RISC-V binaries**, not x86-64 ones.
      We need this because xv6 must be compiled for the **CPU architecture that QEMU is emulating**.

2. Verify toolchain installation
   `riscv64-unknown-elf-gcc --version`
   `qemu-system-riscv64 --version`
   If both commands work, your environment is ready.

3. Clone the xv6 RISC-V Repository
   `git clone https://github.com/mit-pdos/xv6-riscv.git`
   `cd xv6-riscv`

4. Build xv6
   `make`
   Run the command to build the OS. If compilation succeeds, you are ready to run xv6.

5. Run xv6 on QEMU
   Start the xv6 operating system:
   `make qemu`

Appendix B

A list of riscv ISA items and xv6 items that intersect with the xv6 system call implementation — For details of these and more look up Chapter 4 of the xv6 book.

```C/C++
ecall
sret
stvec
sepc
scause
sstatus (SIE and SPP)
sscratch
satp
csrw
a0

TRAMPOLINE
TRAPFRAME
uservec
usertrap
userret
prepare_return
syscall
copyin
copyout
copyinstr
argint
argstr
```